

Big O Notation

HashMap			
Operation	Case	Time Complexity	Explanation
Add	Worst	$O(n^2)$	Rehash and probe up to n times for each of n entries during resize.
	Average	$O(1)$	Few collisions with a good hash function.
	Best	$O(1)$	No collisions; direct insertion.
Remove	Worst	$O(n)$	All entries share the same hash; must probe through all.
	Average	$O(1)$	Few collisions; remove directly.
	Best	$O(1)$	No collisions; direct removal.
Find	Worst	$O(n)$	All entries share the same hash; must probe through all.
	Average	$O(1)$	Few collisions; direct lookup.
	Best	$O(1)$	No collisions; direct lookup.
Resize	Worst	$O(n^2)$	Each entry may need to probe again during rehashing.
	Average	$O(n)$	Few collisions; reinsert n entries.
	Best	$O(n)$	No collisions; direct reinsert.

AVL Tree			
Operation	Case	Time Complexity	Explanation
Add	Worst	$O(\log n)$	Traverse $\log n$ nodes; at most two rotations.
	Average	$O(\log n)$	Self-balancing limits height to $\log n$.
	Best	$O(\log n)$	Still need to traverse to leaf and update balance.
Remove	Worst	$O(\log n)$	Find node, remove, rebalance up to root.
	Average	$O(\log n)$	Traversal and rotations limited by $\log n$ height.
	Best	$O(\log n)$	Even removing leaf needs traversal and balance updates.
Find	Worst	$O(\log n)$	Must traverse root to leaf.
	Average	$O(\log n)$	Binary search through tree.
	Best	$O(1)$	Target found at root.
Calculate Height	All	$O(1)$	Height stored in each node; constant lookup.

Big O Notation

Sorting Algorithms			
Algorithm	Case	Time Complexity	Explanation
Bubble Sort (no opt)	Worst	$O(n^2)$	Every element compared to all others.
	Average	$O(n^2)$	Same as worst on average.
	Best	$O(n)$	Early exit if sorted.
Bubble Sort (with opt)	Worst	$O(n^2)$	Optimization doesn't affect worst case.
	Average	$O(n^2)$	Only improves constant factors.
	Best	$O(n)$	Detect sorted array in one pass.
Insertion Sort	Worst	$O(n^2)$	Reverse-sorted input moves each element.
	Average	$O(n^2)$	Closer to worst on average.
	Best	$O(n)$	Already sorted input.
Selection Sort	All	$O(n^2)$	Find min/max for n elements each iteration.
Cocktail Sort (no opt)	Worst	$O(n^2)$	n swaps for n items.
	Average	$O(n^2)$	Same order as worst case.
	Best	$O(n)$	Early exit if sorted.
Cocktail Sort (with opt)	Worst	$O(n^2)$	Optimization doesn't change worst case.
	Average	$O(n^2)$	Only improves average constants.
	Best	$O(n)$	Detect sorted array in one pass.
Merge Sort	Worst	$O(n \log n)$	Always divides and merges n elements log n times.
	Average	$O(n \log n)$	Always divides evenly.
	Best	$O(n \log n)$	Still must divide and merge.
Quick Sort	Worst	$O(n^2)$	Bad pivots; behaves like selection sort.
	Average	$O(n \log n)$	Average pivot splits input in half.
	Best	$O(n \log n)$	Perfect median pivots.
Heap Sort	All	$O(n \log n)$	n removals from heap, each $O(\log n)$.
LSD Radix Sort	Worst	$O(k n)$	Very long numbers (k digits).
	Average	$O(k n)$	k passes through n numbers.
	Best	$O(k n)$	Small k; few digits per number.

Big O Notation

Pattern Matching				
Algorithm	Scenario	Case	Time Complexity	Explanation
Brute Force	Single Occurrence	Worst	$O(m \cdot n)$	Mismatch at last character each time.
		Best	$O(m)$	Match found on first alignment.
	All Occurrences	Worst	$O(m \cdot n)$	Mismatch at last character each alignment.
		Best	$O(n)$	Immediate mismatches each shift.
Boyer–Moore	Last Occurrence Table	All	$O(m)$	Preprocessing pattern.
	No Occurrences	Worst	$O(m \cdot n)$	Shift by 1 each mismatch.
		Best	$O(m + n/m)$	Pattern shifts by m when chars don't match.
	Single Occurrence	Worst	$O(m \cdot n)$	Match only at end.
		Best	$O(m)$	Match on first alignment.
	All Occurrences	Worst	$O(m \cdot n)$	Frequent single shifts.
		Best	$O(m + n/m)$	Pattern shifts by m each time.
KMP	Failure Table	All	$O(m)$	Preprocessing pass through pattern.
	No Occurrences	All	$O(m + n)$	Scans each char once.
	Single Occurrence	Worst	$O(m + n)$	Handles repeated prefixes efficiently.
		Best	$O(m)$	Perfect match on first alignment.
	All Occurrences	All	$O(m + n)$	Linear scan with preprocessing.
Rabin–Karp	Initial Hashes	All	$O(m)$	Compute pattern and initial text hash.
	No Occurrences	Worst	$O(m \cdot n)$	Hash collisions every time.
		Best	$O(m + n)$	No hash collisions.
	Single Occurrence	Worst	$O(m \cdot n)$	Many hash collisions.
		Best	$O(m)$	Match on first hash comparison.

Big O Notation

	All Occurrences	Worst	$O(m \cdot n)$	Frequent hash collisions.
		Best	$O(m + n)$	Efficient rolling hash with few collisions.

Sorting Properties

Sorting Algorithm	Stable?	Adaptive?	In-Place or Out-of-Place ?	Notes
Bubble Sort	✔ Stable	✔ Adaptive	✔ In-Place	Stops early if no swaps
Selection Sort	✘ Not Stable	✘ Not Adaptive	✔ In-Place	Always scans whole list
Insertion Sort	✔ Stable	✔ Adaptive	✔ In-Place	Excellent for nearly sorted data
Cocktail Shaker Sort	✔ Stable	✔ Adaptive	✔ In-Place	Bidirectional bubble
Merge Sort	✔ Stable	✘ Not Adaptive	✘ Out-of-Place	Needs temp arrays
QuickSort	✘ Not Stable	✘ Not Adaptive	✔ In-Place	Depends on pivot
HeapSort	✘ Not Stable	✘ Not Adaptive	✔ In-Place	Uses heap structure
LSD Radix Sort	✔ Stable	✘ Not Adaptive	✘ Out-of-Place	Requires stable counting sort